4. 函数和调试处理

| 2023-03-04

This document contains Cognex Corporation confidential information, which must be protected and used only in accordance with the Mutual Confidentiality Agreement entered into by Cognex Corporation and Apple Inc. dated November 1, 2014

COGNEX

4.1 函数的定义

```
class Program
{
    static void Write()
    {
        Console.WriteLine("Text output from function");
    }
    static void Main(string() args)
    {
        Write();
        Console.ReadKey();
    }
}
```

- 两个关键字： static 和 void
- 函数名后跟圆括号
- 一个要执行的代码块，放在花括号中

© 2022 Cognex Corporation

4.1.1 返回值

```
class Program
{
    static int SumVals()
    {
        int val1 = 2;
        int val2 = 3
        int sum = val1 * val2;
        return sum;
    }
    static void Main(string() args)
    {
        Console. WriteLine("Summed Values = {sum}");
        Console. ReadKey();
    }
}
```

当函数返回一个值时，可以采用以下两种方式修改函数：

- 在函数声明中指定返回值的类型，但不使用关键字 void 。
- 使用 return 关键字结束函数的执行，把返回值传送给调用代码。

© 2022 Cognex Corporation

4.1.2 参数

```
class Program
{
    static int SumVals(int val1, int val2 )
    {
        int sum = val1 * val2;
        return sum;
    }
    static void Main(string() args)
    {
        int sum = SumVals(2, 3);
        Console. WriteLine("Summed Values = {sum}");
        Console. ReadKey();
    }
}
```

当函数接受参数时，必须指定以下内容：

- 函数在其定义中指定接受的参数列表，以及这些参数的类型。
- 在每个函数调用中提供匹配的实参列表。

© 2022 Cognex Corporation

4.1.2 输出参数 ref 、 out

```
class Program
{
    static void ShowVal(ref int val1, out int val2)
    {
            val1 *= val1;
            val2 = 0;   val2 += 2;
            Console.WriteLine($"val1 = { val1}");
    }
    static void Main(string() args)
    {
            int val1 = 2; int val2;
        ShowVal(ref val1, out val2);
        Console.WriteLine($"val1 = {val1}");
        Console.WriteLine($"val2 = {val2}");
        Console.ReadKey();
    }
    }
```

ref 关键字与 out 关键字的使用方式相同（在函数定义和函数调用中用作参数的修饰符）。但是，二者存在一些重要的区别：

- 把未赋值的变量用作 ref 参数是非法的，但是可以把未赋值的变量用作 out 参数。
- 另外，在函数使用 out 参数时，必须把它看成是尚未赋值。

© 2022 Cognex Corporation

4.2 变量的作用域

```
class Program
{
    int val2 = 2;
    static void ShowVal(ref int val1)
    {
        val1 *= val1;
        Console.WriteLine($"val1 = { val1}");
    }
    static void Main(string() args)
    {
        int val1 = 2;
        ShowVal(ref val1);
        Console.WriteLine($"val1 = {val1}");
        Console.WriteLine($"val2 = {val2}");
        Console.ReadKey();
    }
}
```

全局变量

局部变量

变量的作用域只在定义它的花括号中。

© 2022 Cognex Corporation

4.3 Main 函数

```
class Program
  {
    static void Main(string[] args)
    {
      Console.WriteLine("{0} command line arguments were specified:", args.Length);
      foreach (string arg in args)
      {
        Console.WriteLine(arg);
      }
      Console.ReadKey();
    }
  }
```

Main() 是 C# 应用程序的入口点，执行这个函数就是执行应用程序。这个函数可以返回 void 或 int ，有一个可选参数 string[] args ， Main() 函数可使用如下 4 种版本：

static void Main()

Static void Main(string[] args)

static int Main()

static int Main(string[] args)

Main() 的可选参数 args 是从应用程序的外部接受信息的方法，这些信息在运行应用程序时以命令行参数的形式指定。

© 2022 Cognex Corporation

4.4 结构函数

```
class Program
    {
        static void Main(string[] args)
        {
            CustomerName myCustomer;
            myCustomer.firstName = "John";
            myCustomer.lastName = "Franklin";

Console.WriteLine(myCustomer.Name());
            Console.ReadKey();
        }
        struct CustomerName
        {
            public string firstName,
lastName;
            public string Name()
            {
                return firstName + " " +
lastName;
            }
        }
    }
```

结构函数是一种特别的成员函数。它首要用于为目标分配存储空间，对数据成员进行初始化。关键字 struct 。

nex Corporation

4.5 函数重载

```
class Program
    {
        static void Main(string[] args)
        {
            int[] intArray = { 1, 2, 3, 4 };
            MaxValue(intArray);
        }

        static int MaxValue(int[] intArray)
        {
            return 1;
        }
        static double MaxValue(double[]
doubleArray)
        {
            return 2;
        }
    }
```

函数重载是指在同一个类中定义多个同名的函数，但这些函数的参数类型、参数个数、参数顺序不同，以便应对不同的调用需求。

© 2022 Cognex Corporation

4.6 课堂练习

姓名	分数
小红	78
小明	56
小笑	90
张三	46
李四	66

编写控制台程序按照表格的顺序显示出来并且筛选除分数最高的人单独再显示一次。

© 2022 Cognex Corporation

4.7 Visual Studio 中的调试

1. 非中断 (正常) 模式下的调试
   1) 输出调试信息： Debug.WriteLine(), Trace.WriteLine() 。这两种方法需要使用命名空间 **System.Diagnostice**
   2) 跟踪点 ： 一种把信息输出到 Output 窗口中的办法是使用跟踪点 (tracepoint) 。与 Debug.WriteLine() 相同。
2. 中断模式下的调试
   1) 进入中断模式
      a. 断点：是源代码中自动进入中断模式的一个标记。
      b. Debug.Assert()
      c. Trace.Assert()
   2) 监视变量的内容
   3) 单步执行代码：
      a. Step Into— 执行并移动到下一条要执行的语句上，快捷键 F11 。
      b. Step Over— 同上，但不进入嵌套的代码块，包括函数，快捷键 F10 。
      c. Step Oue— 执行到代码块的末尾处，在执行完该语句块后，重新进入中断模式，快捷键 Shift+F11 。
   4) Immediate 和 Command 窗口
   5) Call Stack 窗口：描述程序是如何执行到当前位置的。

© 2022 Cognex Corporation

4.8 错误处理

1. **Try…catch…finally**
   1) try—— 包含抛出异常的代码。
   2) catch—— 包含抛出异常时要执行的代码。
   3) finally—— 包含始终会执行的代码 ()
   4) 函数 **ThrowException()** ，这个函数会根据调用时使用的参数抛出异常： ThrowException(“none”)— 不抛出异常； ThrowException(“simple”)— 生成一般异常； ThrowException(“index”)— 生成 System.IndexOutOfRangeException 异常； ThrowException(“nested index”)— 包含它自己的 try 块，其中的代码调用 ThrowException(“index”) ，生成一个 System.IndexOutOfRangeException 异常。
2. 列出和配置异常：

   .Net Framework 包含许多异常类型，可以在代码中自由抛出和处理这些类型的异常。 IDE 提供了一个 Exception 窗口，可以检查和编辑可用的异常。
3. 异常处理的注意事项

   如果在当前 catch 块中抛出异常可以使用表达式：

   throw;

   这个表达式会再次抛出 catch 块处理过的异常。如果以这种方式抛出异常，该异常就不会由当前的 try…catch…finally 块处理，而是由上一级的代码处理 (但是嵌套结构中的 finally 块仍会执行) 。

© 2022 Cognex Corporation

4.9 课堂练习

下面定义了一个枚举数据类型 orientation 。编写一个控制台程序，使用结构化异常处理 (SEH) 将 byte 类型的变量安全地强制转换为 orientation 类型变量。注意，可以使用 checked 关键字强制抛出异常，下面是一个示例。在你编写的应用程序中应该使用这段代码。

```
enum Orientation： byte
{
    North = 1，
    South  = 2，
    East = 3，
    West = 4
}
myDirection = checke((Orientation) myByte);
```


© 2022 Cognex Corporation

课后作业

编写控制台程序实现在键盘随意输入一串字符（包含数字、字母、符号）统计数字的数量，字母的数量。

要求：

1 、编辑自定义函数实现，自定义函数中的参数要包含 ref 或者 out 。

2 、使用 try……catch 当输入的字符串包含符号时使用 ThrowException() 函数抛出一般异常，并把异常信息显示到 Dos 窗口。

© 2022 Cognex Corporation

Thank you

© 2022 Cognex Corporation
COGNEX